



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

PERIODO: AGOSTO - DICIEMBRE 2025

Carrera:

Ingeniería en sistemas

Materia:

Patrones de diseño

Tema:

Actividad de evidencia 2_U2 Singleton

Alumnos:

Felix Sandoval Josue Guadalupe 21211940

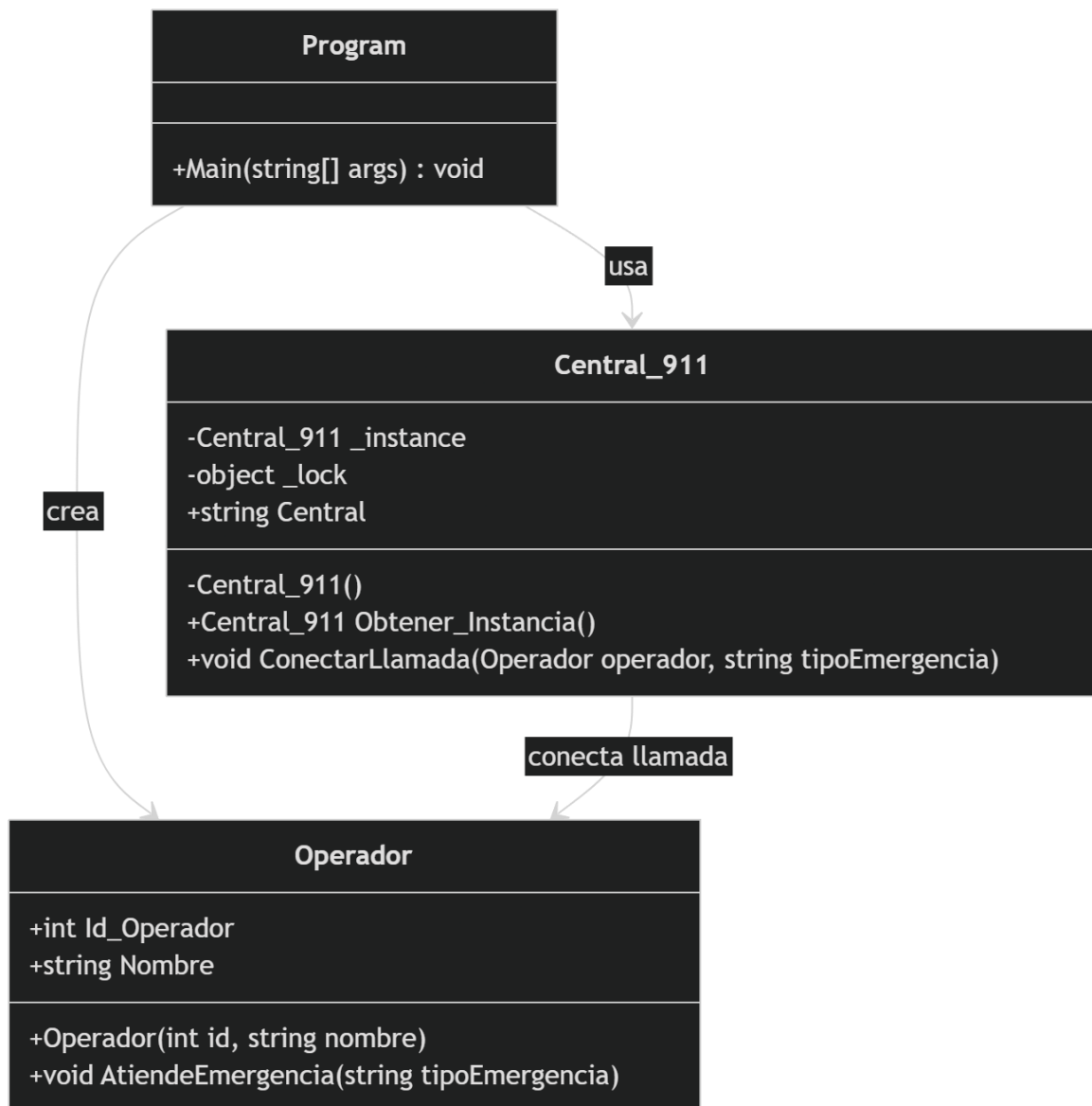
Docente:

Maribel Guerrero Luis

Fecha:

3/9/2026

Diagrama UML



Código ejecutable

```
using System;
```

```
public class Central_911
```

```
{
```

```
    private static Central_911 _instance;
```

```
private static readonly object _lock = new object();
```

```
public string Central { get; private set; }
```

```
private Central_911()
```

```
{
```

```
    Central = "Central 911";
```

```
}
```

```
public static Central_911 Obtener_Instance()
```

```
{
```

```
    if (_instance == null)
```

```
    {
```

```
        lock (_lock)
```

```
        {
```

```
            if (_instance == null)
```

```
            {
```

```
                _instance = new Central_911();
```

```
            }
```

```
        }
```

```
    }
```

```
    return _instance;
```

```
}
```

```
public void ConectarLlamada(Operador operador, string tipoEmergencia)
```

```

{
    Console.WriteLine("\n=====");
    Console.WriteLine("Llamada conectada con el operador: " + operador.Nombre);
    operador.AtiendeEmergencia(tipoEmergencia);
}
}

```

```

public class Operador

```

```

{
    public int Id_Operador { get; set; }
    public string Nombre { get; set; }

```

```

    public Operador(int id, string nombre)

```

```

    {
        Id_Operador = id;
        Nombre = nombre;
    }

```

```

    public void AtiendeEmergencia(string tipoEmergencia)

```

```

    {
        Console.WriteLine($"Operador {Nombre} atendiendo emergencia de tipo:
{tipoEmergencia}");

```

```

        switch (tipoEmergencia)

```

```

        {
            case "Intento de suicidio":

```

```
        Console.WriteLine("Enviando unidades de apoyo psicológico, rescate y patrulla.");
```

```
        break;
```

```
    case "Incendio":
```

```
        Console.WriteLine("Enviando bomberos y unidades de protección civil.");
```

```
        break;
```

```
    case "Accidente":
```

```
        Console.WriteLine("Enviando paramédicos, ambulancia y oficiales de tránsito.");
```

```
        break;
```

```
    case "Violeta":
```

```
        Console.WriteLine("Enviando patrulla especializada en violencia de género.");
```

```
        break;
```

```
    case "Robo":
```

```
        Console.WriteLine("Enviando patrulla y notificando a seguridad pública.");
```

```
        break;
```

```
    case "Asalto":
```

```
        Console.WriteLine("Enviando unidades policiacas de respuesta inmediata.");
```

```
        break;
```

```
    case "Persona sospechosa":
```

```
        Console.WriteLine("Enviando patrulla para verificación de la situación.");
```

```
        break;

    case "Emergencia médica":
        Console.WriteLine("Enviando ambulancia y personal médico.");
        break;

    default:
        Console.WriteLine("Tipo de emergencia no reconocido.");
        break;
    }
}
}
```

```
internal class Program
{
    static void Main(string[] args)
    {
        Central_911 llamada1 = Central_911.Obtener_Instancia();
        Central_911 llamada2 = Central_911.Obtener_Instancia();
        Central_911 llamada3 = Central_911.Obtener_Instancia();

        Operador op1 = new Operador(1, "Laura");
        Operador op2 = new Operador(2, "Carlos");
        Operador op3 = new Operador(3, "Mariana");
        Operador op4 = new Operador(4, "José");
    }
}
```

```

    llamada1.ConectarLlamada(op1, "Incendio");
    llamada2.ConectarLlamada(op2, "Violeta");
    llamada3.ConectarLlamada(op3, "Accidente");
    llamada1.ConectarLlamada(op4, "Intento de suicidio");
    llamada2.ConectarLlamada(op1, "Robo");
    llamada3.ConectarLlamada(op2, "Asalto");
    llamada1.ConectarLlamada(op3, "Persona sospechosa");
    llamada2.ConectarLlamada(op4, "Emergencia médica");

    Console.WriteLine("\n=====");
    Console.WriteLine("Verificación de instancia única:");

    Console.WriteLine("llamada1 == llamada2: " + ReferenceEquals(llamada1,
llamada2));

    Console.WriteLine("llamada2 == llamada3: " + ReferenceEquals(llamada2,
llamada3));

    Console.WriteLine("llamada1 == llamada3: " + ReferenceEquals(llamada1,
llamada3));

    Console.WriteLine("\nNombre de la central: " + llamada1.Central);

    Console.ReadKey();
}
}

```

Lo que agregué fue esto:

Más operadores

- Laura
- Carlos
- Mariana
- José

```
Operador op1 = new Operador(1, "Laura");  
Operador op2 = new Operador(2, "Carlos");  
Operador op3 = new Operador(3, "Mariana");  
Operador op4 = new Operador(4, "José");
```

Más llamadas

- "Incendio"
- "Violeta"
- "Accidente"
- "Intento de suicidio"
- "Robo"
- "Asalto"
- "Persona sospechosa"
- "Emergencia médica"

```
llamada1.ConectarLlamada(op1, "Incendio");  
llamada2.ConectarLlamada(op2, "Violeta");  
llamada3.ConectarLlamada(op3, "Accidente");  
llamada1.ConectarLlamada(op4, "Intento de suicidio");  
llamada2.ConectarLlamada(op1, "Robo");  
llamada3.ConectarLlamada(op2, "Asalto");  
llamada1.ConectarLlamada(op3, "Persona sospechosa");  
llamada2.ConectarLlamada(op4, "Emergencia médica");
```

Verificar que es una única instancia


```
Console.WriteLine("llamada1 == llamada2: " + ReferenceEquals(llamada1,
llamada2));
Console.WriteLine("llamada2 == llamada3: " + ReferenceEquals(llamada2,
llamada3));
Console.WriteLine("llamada1 == llamada3: " + ReferenceEquals(llamada1,
llamada3));
```

Identificar y explicar:

El atributo estático de la instancia es esta línea:

```
private static Central_911 _instance;
```

Explicación:

- static significa que ese atributo pertenece a la **clase** y no a un objeto en particular.
- Su función es **guardar la única instancia** de Central_911.
- Como es estático, todas las partes del programa comparten esa misma variable.
- Al inicio vale null, y cuando se llama a Obtener_Instance(), se crea una sola vez.

Este atributo es el que permite almacenar y reutilizar la única instancia del Singleton.

El constructor es esta parte:

```
private Central_911()
{
    Central = "Central 911";
}
```

Explicación:

- El constructor es el método especial que se ejecuta al crear un objeto.
- En este caso, inicializa la propiedad Central con el valor "Central 911".
- Está declarado como private, lo que significa que **no puede llamarse desde fuera de la clase**.

- Eso impide que alguien escriba `new Central_911()` desde Main u otra clase.

El constructor privado evita que se creen varias instancias y obliga a usar el método `Obtener_Instancia()`.

¿Por qué no se pueden crear múltiples objetos?

No se pueden crear múltiples objetos porque la clase `Central_911` tiene el **constructor privado**.

```
private Central_911()  
{  
    Central = "Central 911";  
}
```

Al ser privado, no permite que desde otras clases se use `new Central_911()`. Eso obliga a que la única forma de obtener un objeto sea mediante el método `Obtener_Instancia()`, el cual crea la instancia solo una vez y después siempre devuelve la misma.

Por eso, el patrón Singleton evita que existan varias instancias de la clase.